

Итоговый отчёт — Smart Parking SPA

1. Описание проекта и актуальность

Проект представляет собой одностраничное приложение для диспетчеризации парковки: визуализация мест, учёт статусов, бронирование и **чат-ассистент** как основной альтернативный канал управления. Актуальность связана с ростом требований к **прозрачности загрузки** и самообслуживанию водителя без отдельного мобильного приложения (веб-first).

2. Архитектурные решения

- **Фронтенд-only MVP:** мок датчиков в браузере снижает стоимость демонстрации и позволяет сфокусироваться на UX и агентной логике.
- **Разделение ответственности:** agentCore отвечает за классификацию текста, handleUserRequest — за сценарии и вызовы сторов, Pinia — за данные и реактивность UI.
- **Расширяемость:** при появлении реального бэкенда сторы становятся адаптерами к REST/WebSocket, а оркестратор — к единому «command bus».

Подробные диаграммы — в ARCHITECTURE.md.

3. Технологии и инструменты (включая ИИ)

Область	Стек
UI	Vue 3 (Composition API), CSS
Состояние	Pinia
Сборка	Vite 6
Маршрутизация	Vue Router (ленивый /admin)
E2E	Playwright
Оффлайн	Ручной public/sw.js + регистрация в production
ИИ в разработке	Промпты для генерации карты, агента, тестов (KEY_PROMPTS.md, PROMPT_LOG.md)

4. Процесс разработки и промпт-инжиниринг

Итеративно уточнялись:

1. **Контракт analyzeIntent** — фиксированные интенты и параметры вместо свободного текста от LLM.
2. **Оптимизация карты** — перенос v-мето на уровень ячейки, а не всей сетки.
3. **E2E-стабильность** — режим localStorage.E2E=1 для детерминированных данных и отключения фоновой симуляции датчиков во время тестов.

Детали — в PROMPT_LOG.md.

5. Тестирование и оптимизация

- **E2E:** 5 сценариев в `e2e/parking-flow.spec.js` (поиск, дешёвое место, вопрос о брони, отмена, админ).
- **Производительность:** разделение чанков (`vue-router`, `pinia`), `v-memo` на ячейках карты, ленивый админ-маршрут.
- **Lighthouse:** инструкция и место для скриншотов — `lighthouse/README.md`.

6. Core Web Vitals (ответ на вопрос блока 3)

Для интерактивной карты и чата наиболее критичны **INP** (отклик на ввод и клики по ячейкам) и **LCP** (скорость появления карты и чата). **CLS** важен из-за динамического появления кнопки «Забронировать» и ответов ассистента — для снижения сдвигов использованы фиксированные области и предсказуемая вёрстка.

Улучшения: мемоизация ячеек, ленивый админ-чанк, ограничение тяжёлых вычислений в обработчиках ввода, контроль размера бандла через `manualChunks`.

7. Вопрос блока 1 (UML / AI-first)

AI-first чат делает поток **недетерминированным** на границе NLU: к UML/DFD добавляются сущности **Intent Router**, **Orchestrator**, **Policy** и опционально **LLM-Proxy** с обратной связью в состояние UI. Классические потоки «форма → валидация → сервис» дополняются веткой «текст → намерение → белый список действий → подтверждение в UI».

8. Вопрос блока 2 (ассистент vs чат-бот OpenAI)

Отличие: у нас **явная** привязка языка к **разрешённым операциям** стор и проверяемым результатам; простой чат-бот к OpenAI чаще генерирует текст без гарантий согласованности с локальными данными. Намерения на клиенте выбраны ради **латентности, тестируемости, экономии и приватности** (не отправляем всю карту и историю во внешнюю модель).

9. Самоанализ и выводы

Как изменилось понимание роли фронтенд-разработчика?

Фронтенд выступает как **оркестратор продуктовой логики** вокруг пользовательского языка: важны контракты, состояние, UX доверия к агенту и наблюдаемость (логи/тесты), а не только визуальные компоненты.

Какие задачи эффективнее с ИИ, какие — без?

- **С ИИ:** каркас компонентов, тесты, документация, разбор рефакторинга.
- **Без / с ручной проверкой:** инварианты бронирования, матрица переходов статусов, безопасность и сценарии отказа.

Этические и технические риски AI-first

- Ошибочное действие агента → нужны **подтверждения** и явные СТА вместо «тихих» мутаций.
- Конфиденциальность переписки при подключении LLM → минимизация PII, свой прокси, политика хранения.
- Зависимость от модели и галлюцинации → **белые списки** команд и валидация на сторе.

Следующий шаг развития

Интеграция с реальными датчиками (WebSocket), серверная очередь событий, CV для номеров, мультиагентная логистика для сети парковок.

10. Демо

- Сайт на Netlify: <https://stepik-ai-course.netlify.app/>
- Видеодемонстрация (скринкаст): [Google Drive](#)